



**WEST UNIVERSITY OF TIMIȘOARA
FACULTY OF COMPUTER SCIENCE
BACHELOR STUDY PROGRAM: COMPUTER
SCIENCE IN ENGLISH**

BACHELOR THESIS

SUPERVISOR:
Conf. Dr. Flavia Micota

GRADUATE:
Bogdan Mitroi

**TIMIȘOARA
2026**

**WEST UNIVERSITY OF TIMIȘOARA
FACULTY OF COMPUTER SCIENCE
BACHELOR PROGRAM: COMPUTER SCIENCE IN
ENGLISH**

Extracting information from open data

SUPERVISOR:
Conf. Dr. Flavia Micota

GRADUATE:
Bogdan Mitroi

**TIMIȘOARA
2026**

Abstract

The physical structure of a city’s street network measurably influences how efficient and sustainable its mobility is, yet the research demonstrating this relationship is usually published as a single scientific analysis rather than as a tool that practitioners can reuse. This thesis presents the design, implementation, and evaluation of a Java desktop application that analyses urban street networks extracted from OpenStreetMap as GraphML files, computes the graph-theoretic and street-level metrics used in the urban mobility literature, enriches them with World Bank socio-economic indicators, and estimates a city’s Urban Mobility Readiness Index (UMRi) together with its Sustainable Mobility (SMi) and Public Transit (PTi) sub-indices. The work builds directly on the study of Sierra-Porta and Herrera-Acevedo (2024), taking its metrics, its sixty-two reference cities, and its regression findings as a specification and turning them into a reusable tool. The application follows a Model–View–Controller architecture with dedicated services for GraphML loading, structural analysis, prediction, World Bank integration, charting, mapping, reporting, and session persistence, and it offers both a tabbed Swing interface and a command-line batch mode. The expensive centrality computations are parallelised across processor cores and split into a fast approximate (Quick) mode and an exact (Full) mode. Evaluated on the full dataset of sixty-two world cities, the independently re-implemented metrics agree closely with the reference study on the quantities that can be checked exactly – a Pearson correlation of 0.997 on graph diameter and 0.78 on clustering coefficient across the fifty matched cities. The prediction engine reproduces the reference regression equation with its fixed published coefficients and additionally fits its own ordinary least-squares (OLS) multiple linear regression models at run time, using the network and socio-economic metrics as predictors to estimate the UMRi/SMi/PTi scores; these models are evaluated by leave-one-out cross-validation, reporting a cross-validated out-of-sample error. The cross-validated models predict the official scores to within roughly seven to nine index points, whereas the fixed published coefficients, applied to the independently re-computed metrics, sit on a different scale and over-predict, which is why the refitted models are recommended in practice. The results indicate that a published urban-mobility analysis can be faithfully re-implemented and packaged into a practical, accessible tool.

Rezumat

Structura fizică a rețelei de străzi a unui oraș influențează în mod măsurabil cât de eficientă și de sustenabilă este mobilitatea acestuia, însă cercetările care demonstrează această relație sunt de obicei publicate ca o analiză științifică punctuală, și nu ca un instrument pe care practicienii să îl poată reutiliza. Această lucrare prezintă proiectarea, implementarea și evaluarea unei aplicații desktop scrise în Java care analizează rețelele stradale urbane extrase din OpenStreetMap sub formă de fișiere GraphML, calculează metricile teoretice de graf și cele la nivel de stradă utilizate în literatura de specialitate privind mobilitatea urbană, le îmbogățește cu indicatori socio-economici de la Banca Mondială și estimează Indicele de Pregătire pentru Mobilitate Urbană (UMRi) al unui oraș, împreună cu subindicii de Mobilitate Sustenabilă (SMi) și Transport Public (PTi). Lucrarea se bazează direct pe studiul realizat de Sierra-Porta și Herrera-Acevedo (2024), preluând metricile, cele șaizeci și două de orașe de referință și rezultatele regresiei acestuia drept specificație și transformându-le într-un instrument reutilizabil. Aplicația urmează o arhitectură de tip Model–View–Controller, cu servicii dedicate pentru încărcarea fișierelor GraphML, analiza structurală, predicție, integrarea cu Banca Mondială, generarea de grafice, hărți, rapoarte și persistența sesiunii, și oferă atât o interfață Swing cu file, cât și un mod de lucru în linie de comandă. Calculele costisitoare de centralitate sunt paralelizate pe nucleele procesorului și împărțite într-un mod aproximativ rapid (Quick) și un mod exact (Full). Evaluată pe întregul set de date format din șaizeci și două de orașe ale lumii, metricile reimplementate independent corespund îndeaproape studiului de referință în privința mărimilor care pot fi verificate exact – o corelație Pearson de 0.997 pentru diametrul grafului și de 0.78 pentru coeficientul de clustering pe cele cincizeci de orașe corelate. Motorul de predicție reproduce ecuația de regresie de referință cu coeficienții ficși publicați și, în plus, antrenează la momentul execuției propriile modele de regresie liniară multiplă prin metoda celor mai mici pătrate (OLS), folosind metricile de rețea și socio-economice ca predictorii pentru a estima scorurile UMRi/SMi/PTi; aceste modele sunt evaluate prin validare încrucișată leave-one-out, raportând o eroare în afara eșantionului. Modelele validate încrucișat prezic scorurile oficiale cu o abatere de aproximativ șapte până la nouă puncte de indice, în timp ce coeficienții ficși publicați, aplicați metricilor recalculat independent, se situează pe o scară diferită și supraestimează, motiv pentru care modelele reantrenate sunt recomandate în practică. Rezultatele indică faptul că o analiză publicată privind mobilitatea urbană poate fi reimplementată fidel și împachetată într-un instrument practic și accesibil.

Statement of Scientific integrity

I declare that this BSc thesis is an original report and represents my own work, conducted under supervision of Flavia Micota. All research methods, data collection, analysis and interpretation presented in this thesis are accurately reported and all sources have been cited appropriately where I have made use of ideas and/or visuals of others.

I used generative AI during the writing and research process. The details are in the Appendix A.

Contents

1	Introduction	9
1.1	Motivation	9
1.2	Objectives	10
1.3	Application Features and Requirements	11
1.3.1	Functional Requirements	11
1.3.2	Non-Functional Requirements	11
1.3.3	Target Users and Use Cases	12
1.4	Thesis Structure	12
2	State of the art	13
2.1	Complex Networks and Graph Theory	13
2.2	Street-Network Metrics for Mobility	13
2.3	The Reference Study	14
2.4	Open Data Sources and Supporting Technologies	15
2.5	Gap Analysis and Thesis Positioning	16
3	System Architecture and Design	17
3.1	Development Methodology	17
3.2	Architectural Overview	17
3.3	Principal Components	19
3.4	Use Case Model	20
3.5	Concurrency Model	20
3.6	Data Flow	22
3.7	Technology Stack	22
4	Implementation	23
4.1	GraphML Loading	23
4.1.1	The GraphML Format	23
4.1.2	File Parsing	24
4.2	Structural Metrics	24
4.3	Prediction Engine	25
4.4	World Bank Integration	27
4.5	Explainability and Interpretation	27
4.6	Charting and Mapping	28
4.7	Export	28
4.8	Session Persistence	30
4.9	Graphical User Interface	30

5	Testing and Evaluation	31
5.1	Testing Methodology	31
5.2	Dataset	31
5.3	Functional Test Results	32
5.4	Correctness Against the Reference Study	32
5.5	Prediction Evaluation	33
5.6	Performance Evaluation	34
5.7	Use Case Walkthroughs	34
5.7.1	Use Case 1: Researcher – Reproducing the Reference Metrics	36
5.7.2	Use Case 2: Student – Comparing Familiar Cities	36
5.7.3	Use Case 3: Analyst – Estimating Readiness for a City . . .	36
5.8	Limitations Observed During Testing	36
6	Discussion	37
6.1	Achievement of Objectives	37
6.2	Relationship to the Reference Study and Existing Tools	37
6.3	Design Trade-offs	38
6.3.1	Quick versus Full Analysis	38
6.3.2	Re-implementation versus Reuse	38
6.3.3	Run-time Regression versus Fixed Coefficients	38
6.3.4	Local Caching and Offline Operation	39
6.4	Limitations and Mitigations	39
6.5	Implications	39
7	Conclusions	41
7.1	Summary	41
7.2	Future Work	42
	Bibliography	44
A	Appendix - Generative AI Tools Usage	45
A.1	Claude (Anthropic) Details	45

Chapter 1

Introduction

1.1 Motivation

Cities are becoming home to the majority of the world’s population. Thus, the issue of how individuals travel within their communities (i.e., mobility) has emerged as one of the most critical issues facing contemporary urban planners. The ability to provide efficient, accessible, and sustainable transportation systems in urban areas represents a significant challenge for both enhancing quality of life and reducing negative impacts on the environment.

The street system of a community, consisting of intersections and the roadways connecting those intersections, does not represent a passive background against which individuals travel but rather actively influences travel behaviour. For example, a dense network of interconnected roads allows travellers to select among numerous paths between destinations and minimizes the distance travelled, whereas sparsely connected road networks require lengthy diversions and force large amounts of traffic onto relatively few major corridors. Therefore, understanding how different types of connectivity impact how well a community supports mobility represents a problem of substantial utility.

Networks illustrating the connectivity of a community’s roadway infrastructure are typically represented as graphs. Complex network analysis has developed a wide range of quantitative measures (e.g., centrality, clustering, diameter, assortativity, etc.) to characterize connectivity characteristics. In addition to providing a framework for characterizing connectivity patterns in road networks, open-source tools have been developed that allow users to convert maps of a roadway system into analysable graphs. Specifically, Boeing et al. [Boe17] demonstrated how the OpenStreetMap (OSM) dataset, which contains detailed information about the geometry of roadways throughout the globe, can be used to create graph representations of the roadway infrastructure of communities. Therefore, combining open roadway data with analytical techniques derived from complex network analysis enables researchers to investigate how mobility is supported by roadway infrastructure in communities around the world.

A research investigation based upon this concept has already been completed. Sierra-Porta and Herrera-Acevedo [SPHA24] investigated the relationship between roadway infrastructure connectivity and mobility preparedness in sixty-two communities identified through the Oliver Wyman Forum’s Urban Mobility Readiness Index. Each community’s roadway network was characterized by computing sev-

eral connectivity metrics. Correlation, Principal Component Analysis (PCA), and multiple linear regression were then employed to identify connectivity attributes (including GDP/capita) that significantly influenced variations in mobility preparedness across communities. As part of this effort, the authors achieved an R-squared statistic (a measure representing the proportion of variance in the dependent variable explained by predictors) equal to approximately 81.5%. Although this research represented a unique investigation employing computational methods implemented within a single Python program, there exists no stand-alone software tool capable of accepting an arbitrarily defined roadway network file(s), calculating corresponding connectivity metrics, comparing roadway networks across communities, integrating socio-economic indicators from international organizations (e.g., World Bank), and estimating how prepared a given community would be relative to other communities when ranked according to its mobility-readiness score.

Therefore, this thesis proposes to fill this void by developing an interactive graphical user interface-based Java desktop application that accepts arbitrary roadway network file(s), calculates connectivity metrics identical to those calculated by Sierra-Porta and Herrera-Acevedo [SPHA24] for roadway networks within communities included in their original study; compares connectivity metrics between communities; integrates various socio-economic indicator values obtained from international organizations (e.g., World Bank); and optionally employs artificial intelligence to recommend strategies for improving mobility-readiness scores in the selected communities.

1.2 Objectives

The central objective of this thesis is to design, implement, and evaluate a desktop application that loads urban street networks from OpenStreetMap-derived GraphML files, computes the graph-theoretic and street-level metrics used in the urban mobility literature, and uses those metrics – combined with socio-economic context – to estimate a city’s Urban Mobility Readiness Index (UMRi) and the related Sustainable Mobility (SMi) and Public Transit (PTi) sub-indices.

This breaks down into several concrete goals. The application must reproduce, on arbitrary input networks, the family of metrics described by Sierra-Porta and Herrera-Acevedo, doing so efficiently enough that large cities with hundreds of thousands of nodes remain tractable. It must let a user compare several cities side by side, both numerically and visually, and place each city in the context of the sixty-two reference cities for which official readiness scores are known. It must enrich purely structural analysis with country-level socio-economic indicators retrieved from an authoritative open source. Finally, it must present all of this through an interface that does not assume the user can program, while still exposing a command-line mode for scripted, reproducible batch processing.

A secondary objective is methodological: to verify that the relationships reported in the reference study survive re-implementation in an independent codebase and toolchain, and to assess how accurately a model trained on the reference cities predicts readiness for cities held out of training.

1.3 Application Features and Requirements

1.3.1 Functional Requirements

The application will include the functionality to read and process a GraphML file that represents a city’s street network and to extract graph structure information, as well as Open Street Map (OSM) attribute information. Additionally, it must calculate two sets of statistics: the first set includes statistical measures of the structure of the street network, including, but not limited to: node count; edge count; normalized betweenness; normalized closeness; degree centrality; graph entropy; diameter; density; clustering coefficient; average shortest path; assortativity; mean degree; reciprocity; and diversity. The second set includes OSM-based statistics related to street characteristics, including, but not limited to: total road length; composition of highways; percentage of one way roads; speed limits; presence of pedestrian/-cycling infrastructure; and the number of intersections (junctions) where multiple streets meet.

In addition to processing these statistics, the application must also provide functionality for comparing/interpreting the output. This includes providing a tabular view of all calculated statistics, which are easily sortable. In addition, this view allows users to export calculated statistics into a CSV file. Users may also elect to create bar charts of selected statistics for comparison among cities. Furthermore, the application provides the capability to visualize the street network via an interactive map, where colors represent OSM attributes such as: highway classification; speed limit; or one-way designation. Finally, the application generates a written comparative report, providing additional context regarding the statistics generated relative to the current mobility research being conducted.

To generate this comparative report, the application retrieves country level socioeconomic data (GDP per capita, urban population percent, CO2 emissions per capita etc.) via an API call to World Bank Data360 [Wor24]. Each city is then associated with its respective socioeconomic data.

Additionally, the application estimates the UMRi metric based upon the structural properties of each loaded city using the previously described sixty-two reference cities as training data. The confidence in the estimated metric is reported. Optionally, if an API key is provided for use with the Google Gemini API [Goo26], a free-form question answering system may be enabled allowing users to ask questions about the loaded cities.

Finally, throughout this process, the application maintains the users work environment - i.e., processed statistics, interface state, report content and advisor history - thus restoring a user’s previous session upon launching the application again.

1.3.2 Non-Functional Requirements

Performance is the predominant non-functional constraint since city networks are large: this thesis used datasets ranging from about twelve thousand nodes/city up to almost one hundred eighty thousand nodes/city. There will need to be a "Quick" analysis mode which can compute answers within seconds to minutes of analysis time for each city by computing samples of the most expensive metrics. Additionally there needs to be a "Full" analysis mode that can compute exact centrality measures

for research purposes but at the cost of potentially longer computing time on the largest graphs. Any long running work must run on background threads so that the user interface remains responsive to the user and report progress to the user.

All costly algorithms – shortest paths for closeness and Brandes algorithm for betweenness – need to be parallelized across available processor cores.

To provide reliability, the application must never modify the input GraphML files used in computation. All computations will operate on in-memory copies of the graph data model.

The code base must clearly separate the data model from the analysis and service layer and the user interface layer. The code base must also have automated unit tests and integration tests.

Finally, the code base must be able to run on any platform with a Java 17 (or later) runtime environment. The application must degrade gracefully when network access is not available for the World Bank or Gemini services.

1.3.3 Target Users and Use Cases

The application is intended for researchers in the field of transportation as well as students and planners interested in analysing and comparing the network structure of street systems without needing to write additional code to analyse the data. Students may use this tool to load several cities they are familiar with, then generate charts illustrating the differences in terms of cluster and connectivity, then review the generated report to learn about the reasons behind why smaller clusters typically demonstrate higher levels of readiness for mobility compared to larger clusters. Urban researchers will be able to apply this tool to reproduce results or extend previous studies by identifying the appropriate folder containing city graph files, selecting the "Full" analysis option, and exporting an exact copy of the metrics table for later statistical processing. Analysts examining specific cities that were not included in the previous rankings may also utilize the tool to calculate the metric values for these cities, retrieve the corresponding World Bank indicator(s), and develop an estimated value of UMRi along with a comparison to other cities within the same group. These examples have been used to identify priority features during development; further details are presented in Chapter 3 and Chapter 5.

1.4 Thesis Structure

The remainder of this thesis is organised as follows. Chapter 2 reviews the background and related work. Chapter 3 presents the system architecture and design. Chapter 4 describes the implementation of the application and its main components. Chapter 5 covers the testing methodology and evaluation results. Chapter 6 discusses the findings, limitations, and comparison with the reference study. Finally, Chapter 7 summarises the main contributions and outlines directions for future work.

Chapter 2

State of the art

This chapter establishes the theoretical and practical context for the thesis. It introduces complex network analysis and the graph metrics relevant to urban street networks, summarises the reference study on which this work is based, surveys the open data sources and supporting tools involved, and identifies the gap that the application developed here is designed to fill.

2.1 Complex Networks and Graph Theory

A complex network is a graph $G = (V, E)$ in which V is a set of vertices and E a set of edges connecting pairs of vertices, used to model a real system whose components interact in non-trivial ways [BLM⁺06]. When the system is a city’s street network, the vertices correspond to intersections and street terminations, and the edges correspond to the road segments between them. Edges may be directed, to capture one-way streets, and may carry weights such as physical length. This representation, popularised for street networks by Boeing’s OSMnx work [Boe17], turns the geometry of a city into an object that decades of graph theory can analyse.

Complex network analysis has been applied across an unusually wide range of disciplines – from protein interaction in biology, through the spread of disease in epidemiology, to risk propagation in financial systems – because the same structural vocabulary describes connectivity, flow, and resilience regardless of what the nodes represent [BLM⁺06]. For urban transport specifically, this conceptual framework enables the formulation of quantitative questions: how directly one area of the city can be reached from another, how evenly traffic-bearing importance is distributed across intersections, and how robust the network is to the loss of a junction.

2.2 Street-Network Metrics for Mobility

The metrics this thesis computes fall into two groups. The first group is purely structural and is derived from the graph itself.

Degree and mean degree. The degree of a node is the number of edges incident to it; in a street network this is roughly the number of streets meeting at an intersection. The average node degree and the closely related average streets-per-node summarise how richly connected the grid is. Higher values indicate more alternative routes and greater redundancy.

Centrality measures. Centrality quantifies the importance of a node within the network [Fre78]. Degree centrality counts direct connections; closeness centrality measures how short the average shortest path from a node to all others is, capturing accessibility; and betweenness centrality measures how often a node lies on shortest paths between other pairs, identifying critical through-routes. Betweenness is the most expensive of these to compute, and the standard efficient method is Brandes’ algorithm [Bra01], which runs in $O(|V||E|)$ time for unweighted graphs.

Clustering coefficient. This measures the degree to which a node’s neighbours are themselves connected, forming tightly knit local groups. In a street network, high clustering reflects dense neighbourhood-scale connectivity that supports short local trips.

Diameter and average path length. The diameter is the longest shortest path in the network, and the average path length is the mean shortest-path distance over all node pairs. Both describe how spread out the network is; compact, well-connected cities tend to have smaller values.

Assortativity. Degree assortativity measures the tendency of nodes to connect to others of similar degree [New02]. In transport networks, high assortativity can indicate segregated clusters that hinder system-wide flow.

Entropy, density, reciprocity, and diversity. Graph entropy captures the disorder of the degree distribution, with lower entropy corresponding to more regular, planned layouts; density is the ratio of actual to possible edges; reciprocity measures how often a directed connection is matched by one in the opposite direction; and diversity captures heterogeneity in local connectivity. Together they give a multi-faceted structural fingerprint of a city.

The second group is street-level and is read from the OpenStreetMap tags attached to edges: total road length, the proportions of each highway class (motorway, primary, residential, footway, cycleway, and so on), the dominant highway type, the share of one-way segments, the distribution of posted speed limits, and the provision for walking and cycling. These do not require graph algorithms but convey practically meaningful information about the character of a city’s streets.

2.3 The Reference Study

The direct foundation of this thesis is the study by Sierra-Porta and Herrera-Acevedo on network structure and urban mobility sustainability [SPHA24]. The authors took the sixty-two cities of the 2024 Urban Mobility City Rankings published by the Oliver Wyman Forum, which scores each city on an Urban Mobility Readiness Index (UMRi) together with a Sustainable Mobility Index (SMi) and a Public Transit Index (PTi). For every city they used OSMnx to build the vehicular street network from OpenStreetMap and computed the structural and street-level metrics described above, alongside socio-economic context such as population, surface area, population density, and GDP per capita.

The three indices are composite scores published by the Oliver Wyman Forum, each placing a city on a roughly 0–100 scale on which a higher value indicates a city better prepared for the future of mobility. They are not computed from the street network: each is a weighted aggregation of a large battery of underlying sub-criteria – covering infrastructure, public-transit provision, sustainability, market

attractiveness, and social impact – that the Forum measures from heterogeneous data sources. Formally, an index I for a city c can be written as a weighted, min–max normalised combination of its p underlying criteria,

$$I(c) = 100 \cdot \sum_{j=1}^p w_j \hat{x}_j(c), \quad \hat{x}_j(c) = \frac{x_j(c) - \min_k x_j(k)}{\max_k x_j(k) - \min_k x_j(k)}, \quad (2.1)$$

where $x_j(c)$ is the raw value of criterion j for city c , the normalisation runs over all ranked cities k , and the non-negative weights satisfy $\sum_j w_j = 1$. The Urban Mobility Readiness Index is the overall, top-level score, $\text{UMRi}(c) = 100 \cdot \sum_{d \in D} \omega_d \hat{S}_d(c)$, in which D is the set of high-level dimensions, $\hat{S}_d(c)$ is the city’s normalised score on dimension d , and ω_d its weight. The Sustainable Mobility Index and the Public Transit Index follow the same form but restrict and re-weight the criteria toward, respectively, environmental sustainability and the quality, coverage, and affordability of public transport; both are themselves dimensions that feed the overall UMRi. The exact criteria and weights are proprietary to the Forum, so this thesis treats the three published scores as ground-truth labels rather than re-deriving them, and instead studies how well the structural form of the street network alone can recover them through regression, as described in Chapters 4 and 5.

Their analysis proceeded in three stages. A correlation analysis, using Kendall’s coefficient to capture non-linear relationships, found that UMRi correlates positively with average node degree, clustering coefficient, and the centrality measures, and negatively with average edge length, graph entropy, diameter, and average path length; the strongest single correlate was GDP per capita at +0.58. A principal component analysis [JC16] showed that the first three components explained roughly sixty-one percent of the variance and that high-UMRi cities cluster together in component space, sharing high node degree, clustering, centrality, and economic resources. Finally, a multiple linear regression of UMRi on the metrics achieved $R^2 = 0.815$ (adjusted 0.755), with GDP per capita, streets per node, graph diameter, clustering coefficient, and assortativity degree emerging as the statistically significant predictors. The overall message is that compact, densely interconnected, well-funded street networks are better prepared for sustainable mobility, a conclusion consistent with the broader compact-city literature [BKK20].

The reference study is a scientific analysis, not a tool. Its computations live in Python notebooks tied to its particular dataset. This thesis takes its metrics, its reference cities, and its regression findings as the specification for a reusable application, and in doing so independently re-implements and re-validates them.

2.4 Open Data Sources and Supporting Technologies

Three open data sources underpin the application. OpenStreetMap provides the raw street geometry and tags for any city in the world under an open licence [Ope26]; in practice these are extracted into GraphML files using OSMnx [Boe17], the de facto standard Python package for acquiring and analysing street networks. The Oliver Wyman Forum’s ranking supplies the official UMRi, SMi, and PTi scores that serve as ground truth for the reference cities. The World Bank’s Data360 API

offers authoritative country-level socio-economic indicators – GDP per capita, urban population share, CO₂ emissions per capita, and more – through a documented REST interface [Wor24].

On the software side, the application relies on the JGraphT library for graph data structures and standard algorithms [JGr26], JFreeChart for the comparative charts, Apache Commons Math for the ordinary least squares regression behind the prediction engine [The24], and Jackson for JSON parsing of both the API responses and the persisted session [Fas26]. The optional AI advisor is built on the Google Gemini API [Goo26]. The geometric and topological classification of road networks by Badhrudeen et al. [BDV⁺22] provides additional context for interpreting structural differences between cities.

2.5 Gap Analysis and Thesis Positioning

The literature establishes both that street-network structure measurably influences urban mobility outcomes and that the metrics needed to quantify it are well understood. What is missing is an accessible, reusable bridge between this body of research and the people who could act on it. The reference study demonstrates the relationships but ships no tool. General-purpose network libraries such as OSMnx or JGraphT compute metrics but require programming and offer no mobility-specific interpretation, no comparison against a corpus of ranked cities, no socio-economic enrichment, and no readiness estimate. General data-visualisation platforms can draw charts but know nothing of street networks or mobility indices.

This thesis fills that gap with a single desktop application that loads city networks from open GraphML data, computes the full set of structural and street-level metrics from the reference study, places each city in the context of the sixty-two ranked reference cities, enriches the analysis with live World Bank indicators, and estimates UMRi, SMi, and PTi from network structure – all behind an interface usable without programming, and backed by a command-line mode for reproducible batch processing. The scope is deliberately bounded to this workflow: load, measure, compare, enrich, predict, and report. The following chapters translate this positioning into a concrete design and implementation.

Chapter 3

System Architecture and Design

This chapter presents the main design choices behind the application: a Java desktop tool that loads OpenStreetMap-derived GraphML files, computes street-network metrics, compares cities, enriches the results with socio-economic indicators, and estimates urban mobility readiness.

3.1 Development Methodology

The application was developed as a data-analysis pipeline with an interactive interface on top. For this reason, its construction followed the CRISP-DM methodology, a standard process model for data-mining and analytics projects [WH00]. CRISP-DM is organised into six iterative phases: business understanding, data understanding, data preparation, modelling, evaluation, and deployment.

In this thesis, the business-understanding phase defined the goal of reproducing and extending selected findings from the reference study and the Oliver Wyman ranking. The data-understanding phase involved collecting the sixty-two city networks with OSMnx and checking which graph attributes and OSM tags were available. Data preparation then transformed the raw GraphML files into structured feature vectors by loading the networks and computing structural and street-level metrics. The modelling phase implemented the three regression-based prediction variants, while the evaluation phase compared their results through leave-one-out cross-validation and against the reference study, as discussed in Chapter 5. Finally, deployment produced the packaged desktop application, including its interface and session persistence.

Figure 3.1 illustrates the CRISP-DM cycle. Its feedback loops were especially relevant in this project, since early metric results often revealed differences from the reference study and required further refinement before the prediction stage could be trusted.

3.2 Architectural Overview

The application follows the **Model–View–Controller (MVC)** pattern. This separates the domain data, the graphical interface, and the coordinating logic, making the system easier to maintain and test. The Swing interface can therefore evolve

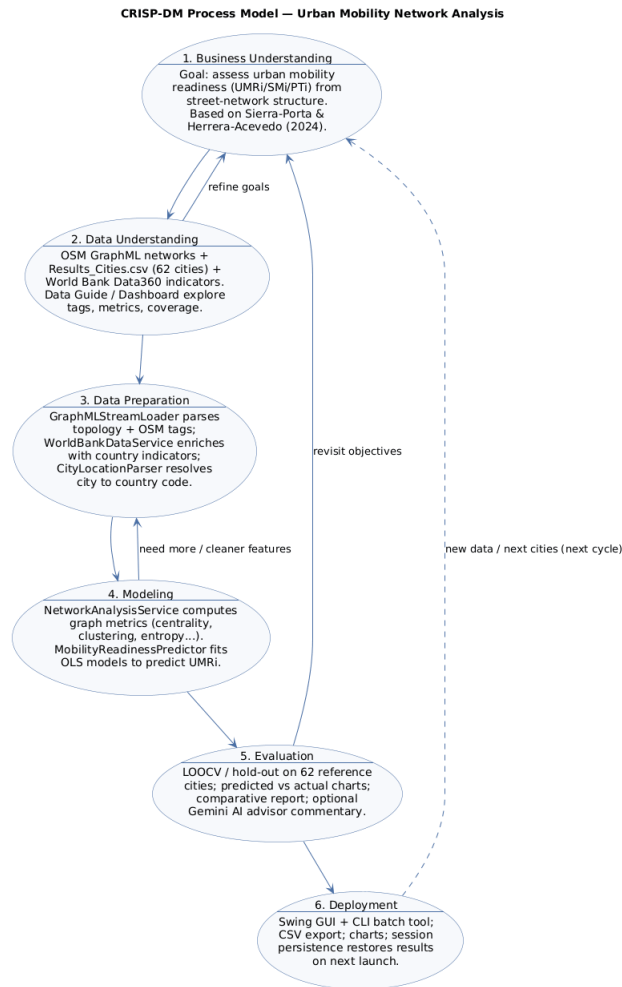


Figure 3.1: The CRISP-DM process model that structured the development of the application.

without affecting the analysis engine, while the services can be tested independently of the user interface.

The code is organised under the `org.example` root into six main packages:

- `org.example.model` – domain objects such as `NetworkMetrics`, `StreetNetworkStats`, `WorldBankCityData`, `ReferenceCityRecord`, and enumerations such as `AnalysisMode` and `PredictionTarget`.
- `org.example.service` – the main business logic, including loading, analysis, prediction, reporting, exporting, API access, and session persistence.
- `org.example.util` – helper classes for reports, charts, statistics, country codes, city parsing, and map support.
- `org.example.view` – the Swing presentation layer, including `ModernNetworkView` and the tab panels.
- `org.example.controller` – the `NetworkController`, which connects the view to the services.
- `org.example.theme` – shared styling and text-rendering utilities.

The entry point is `NetworkAnalysisMain`. With no command-line arguments, it starts the graphical interface. If one or more file paths are provided, it runs in batch mode and exports metrics without opening a window.

3.3 Principal Components

The service layer contains the main functionality of the system. Each service is responsible for a specific part of the workflow:

- `GraphMLLoaderService`, together with `GraphMLStreamLoader` and `GraphMLMetadataParser`, reads GraphML files into JGraphT graphs and extracts OSM metadata, node and edge attributes, and street statistics.
- `NetworkAnalysisService` computes the structural network metrics, including centralities, clustering, diameter, entropy, and assortativity.
- `MobilityReadinessPredictor` estimates `UMRi`, `SMi`, and `PTi` using the reference cities as training data.
- `WorldBankDataService` and `Data360ApiClient` retrieve and cache country-level socio-economic indicators.
- `MetricsExportService` and `MetricsStatisticsService` export results and compute summary statistics across cities.
- `NetworkAnalysisReportService` generates the comparative written report.
- `GeminiInsightService`, through `NetworkInsightFacade`, provides the optional AI advisor.

- `SessionPersistenceService` saves and restores the working state between launches.

The `NetworkController` coordinates these services. It receives user actions from the interface, runs the required service calls in the background, and updates the view when results are ready. Figure 3.2 shows the high-level structure of the system.

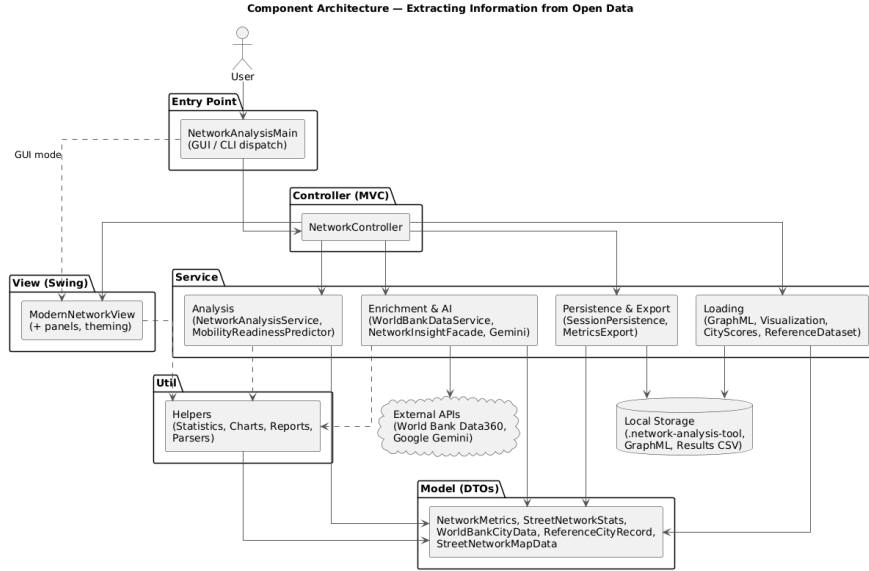


Figure 3.2: High-level MVC component diagram of the application.

3.4 Use Case Model

The application is intended mainly for urban researchers, planning students, and planning analysts. These users interact with a common set of use cases: selecting GraphML files, processing cities in Quick or Full mode, viewing metrics, exporting CSV files, comparing cities through charts, displaying the street map, fetching World Bank indicators, predicting mobility readiness, generating reports, asking the AI advisor, and saving or restoring sessions.

The central use case is *Process Selected Cities*. All other analysis features depend on it, since charts, predictions, reports, and maps are built from the metrics produced during processing. Figure 3.3 presents the corresponding use-case diagram.

3.5 Concurrency Model

Because city street networks can be large, the application uses concurrency both at interface level and at algorithm level. At interface level, long-running tasks such as loading files, computing metrics, fetching indicators, and generating reports are executed outside the Swing Event Dispatch Thread. This keeps the interface responsive while progress is shown to the user. Completed results are then passed back safely using `SwingUtilities.invokeLater()`.

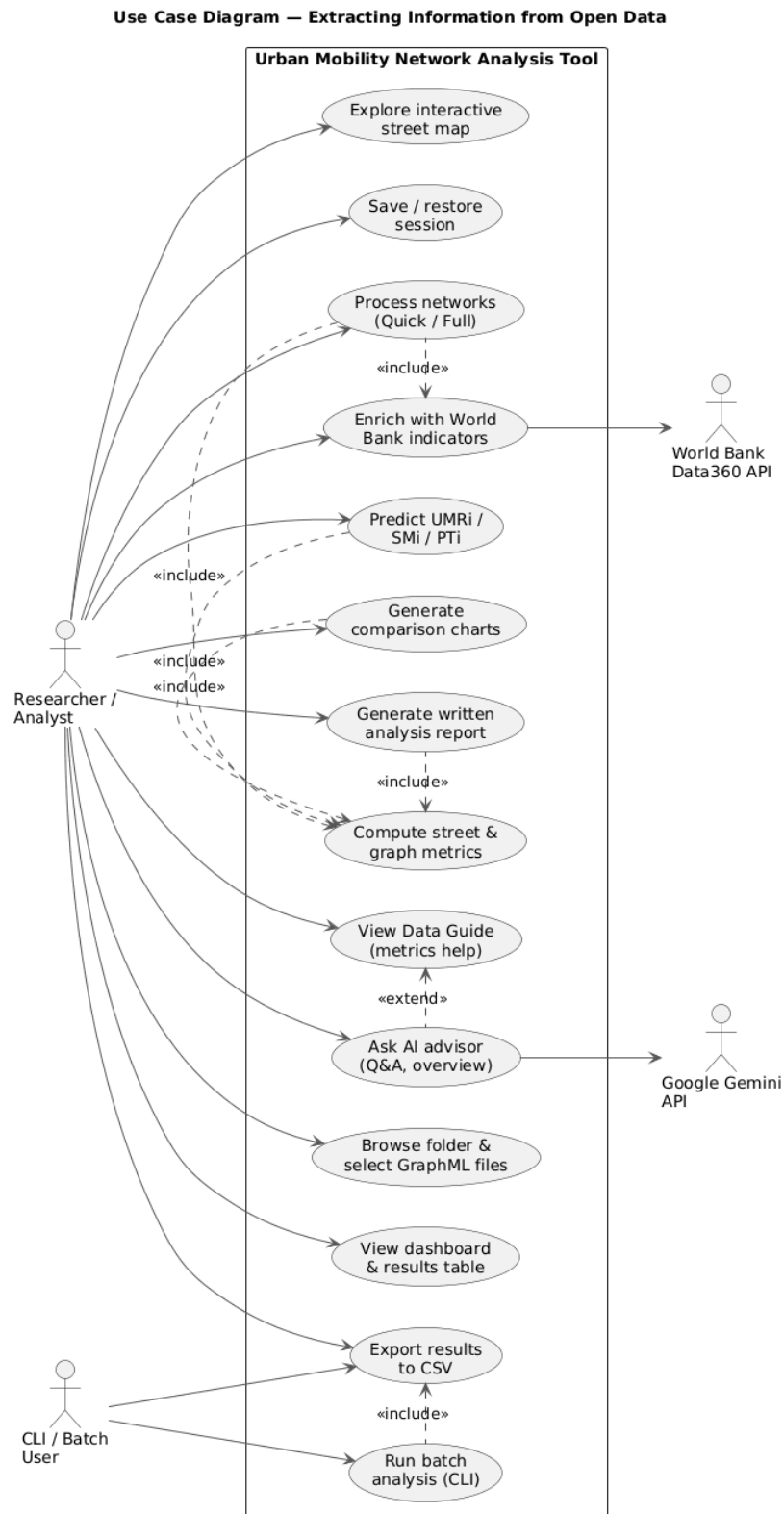


Figure 3.3: Use case diagram for the application.

At algorithm level, `NetworkAnalysisService` uses a fixed thread pool based on the number of available processor cores. The most expensive computations, including all-pairs breadth-first searches and the source-based steps of Brandes’ betweenness algorithm [Bra01], are divided into independent chunks and processed in parallel. Quick mode samples part of the graph to reduce running time, while Full mode performs the exhaustive computation needed for exact research results.

3.6 Data Flow

A typical analysis session follows this sequence:

1. The user selects one or more `.graphml` files in the **Files** tab and chooses Quick or Full analysis.
2. `GraphMLLoaderService` loads each file into a graph and extracts its OSM metadata and street statistics. The original file is not modified.
3. `NetworkAnalysisService` computes the structural metrics for each city.
4. `NetworkController` stores the results and updates the Dashboard, Results table, and related tabs.
5. When requested, the application fetches World Bank indicators, predicts the UMRi, generates charts, displays the coloured street map, or writes the report.
6. The metrics can be exported to CSV, and the full session can be saved and restored automatically.

3.7 Technology Stack

The project uses Apache Maven for dependency management. The main third-party libraries are:

- **JGraphT** [JGr26] – graph structures and GraphML import support.
- **JFreeChart** – comparative charts for the interface and reports.
- **Apache Commons Math** [The24] – ordinary least squares regression and descriptive statistics.
- **Jackson** [Fas26] – JSON parsing and session serialisation.
- **Google Gemini API** [Goo26] – the optional natural-language advisor.
- **OpenCSV** and **SLF4J** – CSV export and logging.

The application targets **Java 17 and above**. By default, it compiles with release 21, while a Java 17 Maven profile is also available. Since the interface is built with Swing, the application runs on Windows, macOS, and Linux without platform-specific changes.

Chapter 4

Implementation

This chapter details the implementation of the application’s major subsystems: GraphML loading and metadata parsing, structural metric computation, the prediction engine, the World Bank integration, charting and mapping, export, and session persistence.

4.1 GraphML Loading

The application consumes city street networks in the GraphML format, an XML-based language for describing graphs that has become a common interchange format in network science

4.1.1 The GraphML Format

A GraphML document is a plain-text XML file whose root **graphml** element contains a single **graph** element; the graph in turn holds a flat list of **node** elements and **edge** elements. Each node carries a unique identifier and each edge names the identifiers of the source and target nodes it connects, so the topology of the network – which intersections are joined by which road segments – is fully described by these references. Crucially, GraphML is not limited to bare topology: it supports typed attributes through **key** declarations, each of which defines a named property (with a data type and a target of either **node**, **edge**, or **graph**), and **data** elements inside nodes and edges that bind a value to one of those keys. This attribute mechanism is what makes GraphML suitable for street networks, because it lets the file carry, alongside the graph structure, the full richness of the OpenStreetMap tags attached to every intersection and every road.

The files used in this thesis are produced by OSMnx [Boe17], which downloads the drivable street network of a named place from OpenStreetMap and serialises it to GraphML with a well-defined schema. In these files each **node** is an intersection or a dead-end and carries, through its **data** elements, its geographic coordinates as latitude (**y**) and longitude (**x**), an OSM identifier, and a **street_count** giving the number of streets incident to that point. Each **edge** is a directed road segment between two intersections and carries a far larger set of OSM tags: the **highway** class that records the road’s function (**motorway**, **trunk**, **primary**, **secondary**, **tertiary**, **residential**, **living_street**, **footway**, **cycleway**, and so on), the physical **length** of the segment in metres, the **oneway** flag, the posted **maxspeed**, the number of

`lanes`, `name` and `ref` labels, `bridge` and `tunnel` markers, and an optional `geometry` string giving the exact polyline of the road. The `graph` element itself declares whether the edges are directed – for OSMnx drive networks they are, so that one-way streets are represented faithfully – and may carry graph-level metadata such as the CRS and the source query. Understanding this schema is what allows the loader to extract meaningful mobility information rather than just an abstract graph.

4.1.2 File Parsing

Loading proceeds in two cooperating steps over a single read of the file, so that even the largest networks are parsed in one pass without holding the whole XML tree in memory, driven by a streaming (StAX) XML reader that emits start- and end-element events as it walks the document. The first step builds the topology: as the reader encounters each `node` it adds a vertex to a JGraphT directed graph keyed by the node identifier, and as it encounters each `edge` it adds the corresponding directed edge between the two referenced vertices. This JGraphT graph is the structure on which all the graph-theoretic metrics are later computed, and it deliberately discards the geometry and descriptive tags, keeping only what the algorithms need. The second step, performed concurrently in the same pass by a metadata aggregator, reads the `key` schema once to learn which attribute each key identifies and then, for every node and edge `data` element, accumulates the street-level statistics: it sums the `length` of every segment for total road length, tallies the count and total length of each `highway` class so their percentage mix and dominant class can be reported, counts one-way segments for the one-way share, accumulates speed and lane figures, counts bridges and tunnels, and tracks the bounding box of node coordinates for the map. The expensive `geometry` payloads are detected and skipped rather than parsed, since the metrics need only whether geometry is present, and the result is a populated `StreetNetworkStats` object beside the graph. Filenames follow the convention `City_Country.graphml`, from which the city and country names are parsed so the city can later be matched to its World Bank country code and reference ranking entry. Throughout, the original file is opened read-only and never altered, satisfying the requirement that input data is never modified.

4.2 Structural Metrics

`NetworkAnalysisService` computes the graph-theoretic metrics for a loaded network. Its public `calculateMetrics` method takes the graph, the source filename, and an `AnalysisMode` (Quick or Full), and returns a populated `NetworkMetrics` object. The service first records the node count, edge count, and directedness, then dispatches the independent computations concurrently onto a fixed thread pool.

Degree-based metrics – mean degree, average degree centrality, graph entropy from the degree distribution, and graph density – are computed in a single pass over the vertex set. The clustering coefficient is computed by examining the connectivity among each node’s neighbours, and reciprocity by counting matched directed edge pairs. The two expensive families of metrics are handled by partitioning the source vertices into chunks processed in parallel. A breadth-first search from each source yields the data for closeness centrality, average path length, and diameter, which are then aggregated across chunks; betweenness centrality is accumulated per source

using Brandes’ algorithm [Bra01] and normalised at the end. Listing 4.1 sketches the parallel dispatch.

Listing 4.1: Parallel dispatch of the structural metric computations

```

1 CompletableFuture<Void> degreeTask = CompletableFuture.runAsync(
2     () -> computeDegreeMetrics(graph, metrics, n, m, isDirected), executor);
3 CompletableFuture<Void> clusterTask = CompletableFuture.runAsync(
4     () -> computeClustering(graph, metrics), executor);
5 CompletableFuture<Void> reciprocityTask = CompletableFuture.runAsync(
6     () -> computeReciprocity(graph, metrics, m, isDirected), executor);
7
8 for (int c = 0; c < chunks; c++) {
9     final int from = chunkStart[c], to = chunkStart[c + 1];
10    brandFutures.add(executor.submit(
11        () -> brandesChunk(graph, vertices, vertexIndex, from, to, n, isDirected
12        ↪ )));
13    bfsFutures.add(executor.submit(
14        () -> bfsChunk(graph, vertices, from, to, n, isDirected)));
15 }
16 CompletableFuture.allOf(degreeTask, clusterTask, reciprocityTask).join();
17 aggregateBFSResults(bfsFutures, metrics, n);
18 aggregateBrandesResults(brandFutures, metrics, n, isDirected);

```

The distinction between the two analysis modes lives here. When the graph is large – above an internal threshold on node or edge count – and the mode is Quick, the service takes an approximate path: betweenness is skipped, closeness and path-length statistics are estimated from a bounded sample of source vertices, clustering is omitted for the very largest graphs, and a flag on the resulting `NetworkMetrics` records that the values are approximate. In Full mode the exhaustive computation is always used regardless of size, so that researchers obtain exact figures at the cost of longer running time.

4.3 Prediction Engine

The prediction subsystem estimates a city’s UMRi from its structural metrics, reproducing and extending the regression analysis of the reference study [SPHA24]. It is built on the sixty-two reference cities, loaded by one component that reads their metrics and another that reads their official scores. Conceptually, every model in the engine is a linear predictor: given a feature vector $\mathbf{x} = (x_1, \dots, x_k)$ extracted from a city’s network, the estimated index is

$$\hat{y}(\mathbf{x}) = \beta_0 + \sum_{i=1}^k \beta_i x_i, \quad (4.1)$$

where β_0 is an intercept and the β_i are the per-feature coefficients. What distinguishes the three model types the engine offers is where those coefficients come from and how the features are scaled before equation (4.1) is applied.

The first model, the paper linear model, applies the fixed regression coefficients reported in the reference study directly to the five predictors clustering coefficient, assortativity degree, streets per node, graph diameter, and GDP per capita, as shown in Listing 4.2. It uses no intercept and no scaling, so it reproduces exactly the published equation and serves as a faithful re-statement of the reference regression.

Listing 4.2: Fixed-coefficient model from the reference study

```

1 private static double predictPaper(double[] f) {
2     return 190.4 * f[0]    // clustering coefficient
3     - 45.2 * f[1]        // assortativity degree
4     + 29.1 * f[2]        // streets per node
5     - 0.05 * f[3]        // graph diameter
6     + 0.0002 * f[4];    // GDP per capita
7 }

```

The second and third model types, the structural OLS model and the full OLS model, fit an ordinary least squares regression on the reference cities at run time using Apache Commons Math’s `OLSMultipleLinearRegression` [The24]. Ordinary least squares chooses the coefficient vector β that minimises the sum of squared residuals over the n training cities, $\beta = \arg \min_{\beta} \sum_{c=1}^n (y_c - \hat{y}(\mathbf{x}_c))^2$, whose closed-form solution is $\beta = (X^\top X)^{-1} X^\top \mathbf{y}$ for the design matrix X of stacked feature vectors and the label vector $\mathbf{y}$ of official scores. The structural model uses the four purely topological predictors (clustering coefficient, assortativity degree, streets per node, graph diameter); the full model adds graph density, degree centrality, average path length, and GDP per capita. In the full model the features are standardised to zero mean and unit variance before fitting, $\hat{x}_i = (x_i - \mu_i)/\sigma_i$, so that the coefficients are directly comparable in magnitude despite the very different natural units of, say, a clustering coefficient and a GDP figure in dollars; the same stored means and standard deviations are reapplied when the fitted model is later used to score a loaded city.

The way the program turns a user’s loaded city into a predicted UMRi is therefore a short pipeline. When the user opens the Prediction tab and selects a target index and a model, the engine assembles the training set from the sixty-two reference cities, extracting for each the features the chosen model expects and reading its official score as the label, and fits the model once on all sixty-two cities to obtain the coefficients shown in the Regression coefficients panel. Then, for every loaded and processed city, it extracts that city’s feature vector from its computed `NetworkMetrics` – clustering, assortativity, diameter and the rest from the structural metrics, streets-per-node from the OSM street statistics, and GDP per capita from the attached World Bank data, falling back to the reference value or the dataset median when a live figure is unavailable – scales it if the model requires, and evaluates equation (4.1) to produce the estimated score in the Predicted column. Where the loaded city is itself a reference city, its official score appears in the Actual column and the signed residual in the Error column, exactly as the screenshots of the Prediction tab in Chapter 5 show.

To report how much confidence the user should place in a prediction, each OLS model is additionally evaluated by leave-one-out cross-validation: for every reference city in turn the model is refitted on the other sixty-one cities and used to predict the held-out city, from which the cross-validated coefficient of determination R^2 , the mean absolute error, and the root-mean-square error are computed and displayed alongside the predictions. This separates the optimistic in-sample fit that the paper model reports from the cross-validated out-of-sample error that tells the user how the model behaves on a city it has never seen.

4.4 World Bank Integration

Socio-economic context is retrieved from the World Bank Data360 API [Wor24]. `CountryCodeResolver` maps each city’s country name to its ISO country code, and `Data360ApiClient` requests the latest available value of each indicator – GDP per capita, urban population share, CO₂ emissions per capita, population, and others enumerated in `WorldBankIndicators` – for that country. Responses are parsed with Jackson and cached as one JSON file per indicator under the user’s home directory, so repeated runs and offline use do not re-hit the API; an offline flag forces cache-only operation. The retrieved values are attached to the corresponding cities as `WorldBankCityData` and feed both the World Bank tab and the GDP-dependent prediction models.

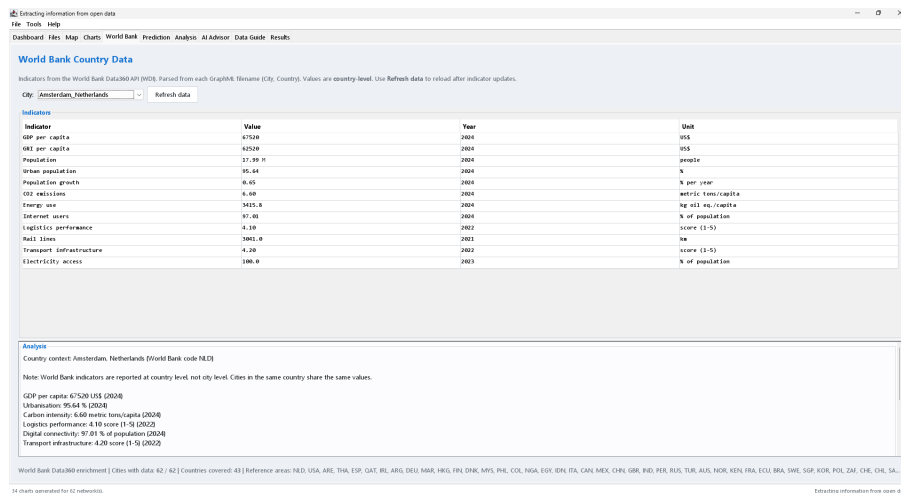


Figure 4.1: The World Bank tab, showing the country-level Data360 indicators retrieved for a selected city together with the generated socio-economic summary.

4.5 Explainability and Interpretation

A recurring difficulty with structural-mobility analysis is that its outputs – a clustering coefficient of 0.21, an assortativity of -0.18 , a predicted UMRi of 58.8 – are opaque to anyone not already fluent in network science. A central design concern of the application is therefore explainability: every number it produces is accompanied, somewhere in the interface, by a plain-language account of what it means and why it matters for mobility. This is served by three cooperating components that together form the explanatory layer of the tool.

The first is the written analysis report. When requested, the report service assembles a structured narrative over the loaded cities: it summarises each city’s structural profile, ranks the cities against one another, relates the observed metric values to the mobility findings of the reference study, and flags the cities that stand out as particularly compact or particularly sparse. Rather than dumping raw figures it frames them – explaining, for instance, that a high streets-per-node value indicates a richly interconnected grid with many alternative routes, and tying that back to why such cities tend to score higher on readiness. The report is deterministic

and works entirely offline, so the interpretive layer is always available even without any external service.

The second component is the Data Guide, a dedicated tab that documents the OSM tags and every computed metric in accessible terms. For each metric it gives a one-line definition, an indication of the direction in which it relates to mobility (whether higher or lower is generally better), and the typical range observed across the reference cities, so that a user reading a value off the results table can immediately judge whether it is high or low and what that implies. This turns the tool into something that teaches its own vocabulary, which directly supports the student use case of Chapter 5.

The third and most flexible component is the optional AI advisor, which provides free-form, conversational explanation on top of the deterministic layers. It is built on the Google Gemini API [Goo26] and is engaged only when the user supplies their own API key, so the tool never depends on it. When the user asks a question, the advisor does not send the raw GraphML or the entire metric table to the model; instead an insight facade first builds a compact, focused context from the loaded cities – the relevant metrics, the best and worst performers, and the official scores where known – and the request is issued under a fixed system prompt that casts the model as an urban-mobility analyst, instructing it to interpret the specific graph-theoretic metrics and OSM infrastructure data in the context of the UMRi, to reference the reference study where relevant, and to stay concrete and comparative. Beyond free questioning, the facade exposes higher-level operations such as generating a comparative overview of all loaded cities or contrasting the strongest and weakest city. The client is defensive about the realities of a remote model – retrying transient rate-limit and overload responses, falling back to alternative model versions when the requested one is unavailable, and surfacing clear error messages when a key is missing or rejected – so a failure in the optional advisor never compromises the core, fully explainable analysis delivered by the report and the Data Guide.

4.6 Charting and Mapping

Comparative charts are produced by `NetworkChartGenerator` using `JFreeChart`. Given a set of loaded cities and a chosen metric, it builds a category dataset and renders a side-by-side bar chart, allowing structural metrics or street statistics to be compared visually across cities. The interactive map is drawn by `StreetNetworkMapPanel`, which projects the edge geometry from the loaded network and colours each segment according to a selectable `OpenStreetMap` attribute – highway type, speed limit, one-way status, or bridge/tunnel – using the palette defined in `HighwayColorScheme`. A spatial index (`SegmentSpatialIndex`) supports efficient hit-testing so that segments can be selected and inspected.

4.7 Export

`MetricsExportService` writes the computed metrics for all loaded cities to a CSV file, one row per city and one column per metric, covering both the structural metrics and the street statistics. In command-line mode the same service is in-

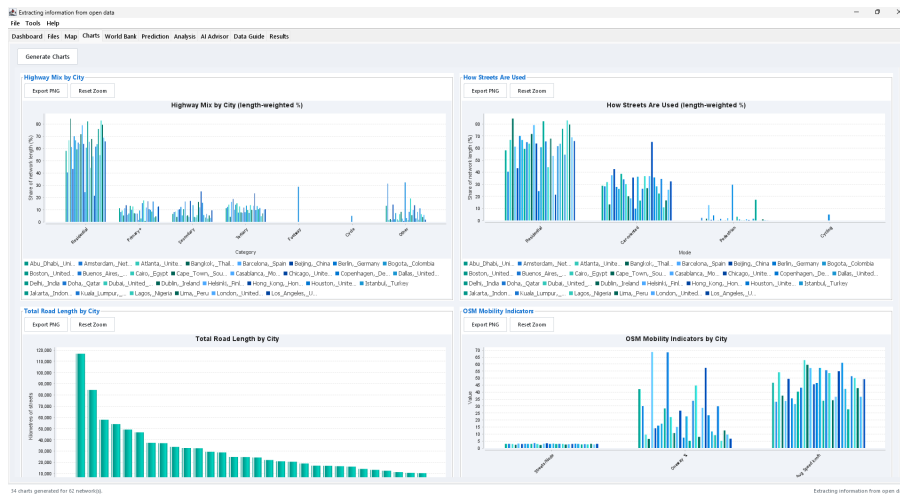


Figure 4.2: The Charts tab comparing the highway mix, street usage, total road length, and OSM mobility indicators across the loaded cities.

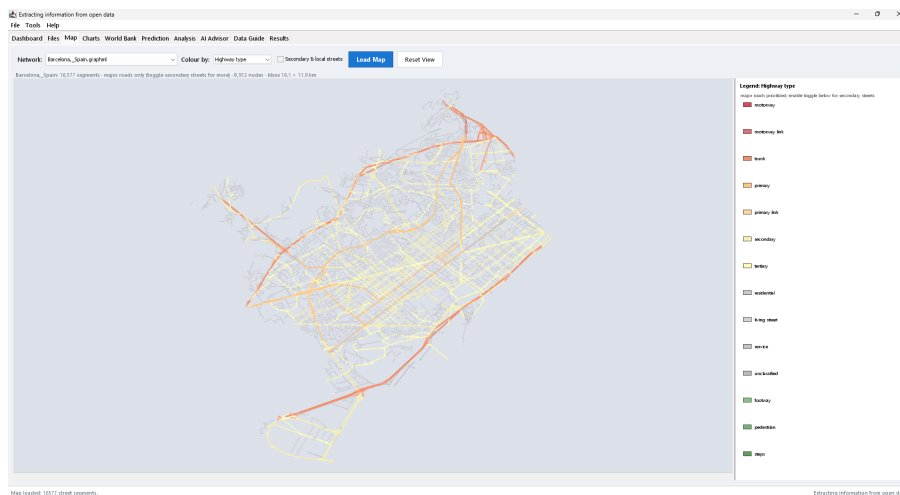


Figure 4.3: The Map tab rendering a city's street network, with each segment coloured by OpenStreetMap highway type.

voked automatically, exporting an `_all_metrics.csv` file next to each processed input. `MetricsStatisticsService` computes cross-city summary statistics (minimum, maximum, mean, and standard deviation per metric) used in the report and the dashboard.

4.8 Session Persistence

To spare the user from reprocessing on every launch, `SessionPersistenceService` serialises the working state to a JSON file in the user’s home directory using Jackson [Fas26]. The saved session includes the processed metrics for each city, the selected folder path, the generated analysis text, the World Bank data, and the AI advisor conversation history. A backup of the previous session is kept alongside the current one. On startup the controller restores this state, so the application reopens exactly as the user left it.

4.9 Graphical User Interface

The interface is built with Java Swing and organised as a tabbed window managed by `ModernNetworkView`, with consistent styling provided by the `org.example.theme` package. The tabs map directly onto the workflow: **Dashboard** shows key street indicators; **Files** browses and processes GraphML files; **Map** renders the coloured network; **Charts** compares metrics across cities; **World Bank** shows the country indicators; **Prediction** estimates the UMRi; **Analysis** presents the report; **AI Advisor** hosts the optional Gemini chat; **Data Guide** explains the OSM tags and metrics; and **Results** lists every computed value in a sortable, exportable table. All updates triggered by background work are dispatched through `SwingUtilities.invokeLater()` to respect Swing’s single-threaded model.

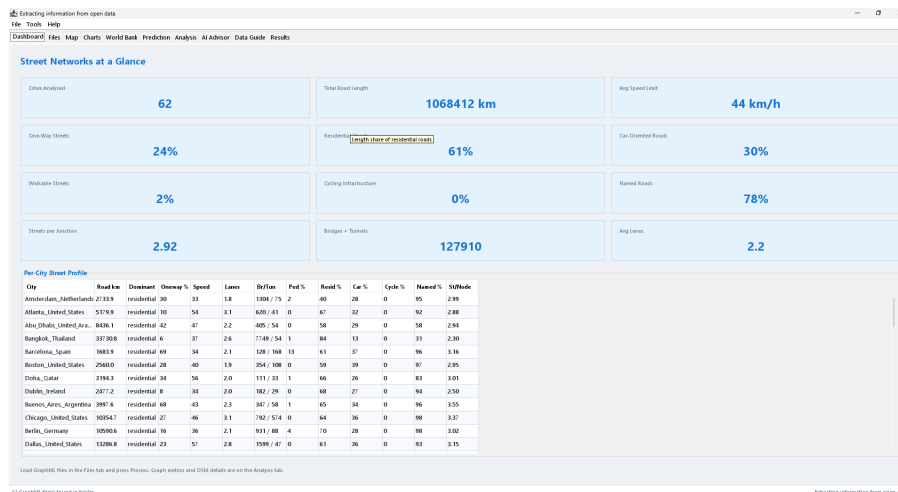


Figure 4.4: The Dashboard tab, summarising the loaded cities through aggregate street indicators and a per-city street profile.

Chapter 5

Testing and Evaluation

This chapter describes how the application was tested and evaluated: the testing approach, the dataset, the functional, correctness, and performance results, and a few use-case walkthroughs.

5.1 Testing Methodology

Testing combined automated tests with manual evaluation against the reference study, on four levels.

- **Unit tests** – service and utility classes were tested in isolation with JUnit 5, covering city-location parsing (`CityLocationParserTest`), country-code resolution (`CountryCodeResolverTest`), GraphML metadata aggregation (`GraphMLMetadataParserAggregatorTest`), the prediction engine (`MobilityReadinessPredictorTest`), CSV export (`MetricsExportServiceTest`), cross-city statistics (`MetricsStatisticsServiceTest`), the spatial index (`SegmentSpatialIndexTest`), and session persistence (`SessionPersistenceServiceTest`).
- **Integration tests** – whole flows were run on real city files, including the round trip that saves a session with World Bank data and restores it (`SessionWorldBankRoundTripTest`).
- **Correctness evaluation** – the metrics and predictions were compared against the values reported in the reference study [SPHA24] for the cities the two have in common.
- **Performance evaluation** – the full dataset was processed in both Quick and Full modes to measure running time and the behaviour of the two paths on large graphs.

5.2 Dataset

The dataset is the set of sixty-two cities from the 2024 Urban Mobility City Rankings, the same corpus the reference study used, extracted from OpenStreetMap into

GraphML with OSMnx [Boe17]. The cities cover five world regions, as shown in Table 5.1. Because the official UMRi, SMi, and PTi scores are known for each city, the dataset serves both as training data for the prediction engine and as ground truth for evaluating it.

Region	Cities
Europe	18
Asia	14
North America	12
Latin America	9
Middle East and Africa	9
Total	62

Table 5.1: Regional distribution of the evaluation dataset.

The networks differ a lot in size, from about twelve thousand nodes for the smallest cities to over four hundred thousand for the largest (Cape Town), with Tokyo, Riyadh, Melbourne, and Istanbul all above two hundred thousand. This spread is what makes the Quick/Full distinction worthwhile in practice.

5.3 Functional Test Results

All sixty-two GraphML files loaded and processed without error. The structural metrics and street statistics were filled in for every city, the results table and dashboard showed the values correctly, CSV export reproduced them, the comparative charts rendered, and the interactive map drew each network coloured by the chosen OpenStreetMap attribute. The street statistics matched what one would expect: **residential** is the dominant highway class for fifty-nine of the sixty-two cities, with a few dominated instead by **secondary**, **tertiary**, or **footway** segments.

The automated suite was run in the environment given in Table 5.2.

Item	Value
Java (JDK) version	21
JUnit version	5
Operating system	Windows 11
Build / test runner	Maven (<code>mvn test</code>)

Table 5.2: Environment in which the automated test suite was executed.

In total **11** test cases were executed across the service and utility layer, and all **11** passed. They exercise the parsing, resolution, aggregation, prediction, export, statistics, spatial-index, and persistence components, plus the session round trip on real city files.

5.4 Correctness Against the Reference Study

Since the application re-implements in Java the metric computation that the reference study did in Python, the most useful correctness check is to compare the

two on the cities they share. The comparison only covers the cities matched by name between the application’s output and the published dataset: of the sixty-two, fifty matched, and the other twelve could not be aligned automatically because of different spellings, accented or transliterated names, and country qualifiers in the filenames. Those twelve are not a computational failure – their metrics were computed normally – they are just left out here because there is no reference value to compare against.

Two of the reported metrics allow a clean numerical comparison: graph diameter and the clustering coefficient. Diameter is an exact integer and does not depend on vertex sampling, so it gives the cleanest cross-implementation check; on the fifty matched cities the two implementations agree almost exactly, with a Pearson correlation of $r = 0.997$. The clustering coefficient is reported where the application computed it exactly (Quick mode skips it on the largest graphs), and there the agreement is good but lower, $r = 0.78$, reflecting how neighbour connectivity is handled on directed graphs and the approximation used on the largest networks. The other published quantities were not compared one-to-one, since the reference study does not report them per city in a recoverable form or their definitions differ enough that a raw correlation would not mean much. Within this scope, the strong diameter and good clustering agreement show that the re-implementation reproduces the reference metrics on the quantities that can be checked exactly; a full metric-by-metric validation is left for future work.

5.5 Prediction Evaluation

The prediction engine was evaluated by leave-one-out cross-validation over the sixty-two reference cities, as described in Chapter 4. The fixed-coefficient paper model reproduces the published *regression equation* – which reported a fit of $R^2 = 0.815$ on UMRi [SPHA24] – by applying its coefficients unchanged, but reproducing the equation is not the same as reproducing its accuracy on metrics from a different implementation. The run-time OLS models are instead refitted on the reference cities, trading some in-sample fit for a cross-validated out-of-sample estimate; the full model, which adds GDP per capita to the topological predictors, predicts held-out cities clearly better than the purely structural one, in line with the reference finding that GDP per capita is the strongest single correlate of mobility readiness (+0.58). Each prediction is reported with its cross-validated R^2 , mean absolute error, and root-mean-square error.

To compare the three methods directly, their predicted UMRi values were set against the official 2024 scores. The dataset has official scores for all sixty-two cities, but this head-to-head only uses those where a model produced a prediction and a reference label was available after filename matching: thirty-five cities for the two OLS models and thirty-four for the paper model. Table 5.3 gives the error magnitudes, and Figure 5.1 shows the Prediction tab for each method. The two OLS models are clearly more accurate, with mean absolute errors of 7.4 (full) and 8.9 (structural) index points and only a small positive bias. The paper model over-predicts every city, with a mean absolute error above thirty points – not because the published equation is wrong, but because its coefficients are tied to the metric definitions, scaling, and units of the original Python pipeline and no longer sit on the

right scale once applied to metrics from a different implementation. For networks processed by this application, then, the refitted models – especially the full one – should be preferred, with the paper model kept as a faithful re-statement of the original equation rather than the recommended predictor.

Method	Cities	MAE	RMSE	Mean error
Structural OLS	35	8.9	10.6	+2.9
Full OLS	35	7.4	8.4	+3.0
Paper linear model	34	30.2	31.0	−30.2

Table 5.3: Accuracy of the three UMRi prediction methods against the official 2024 scores on the matched reference cities. MAE and RMSE are in index points; the mean error is the average signed residual (actual minus predicted).

5.6 Performance Evaluation

Performance was measured by processing the full dataset in both modes on the hardware in Table 5.4.

Item	Value
CPU model / cores	AMD Ryzen AI 7 350 / 8
Physical / logical cores	8 / 16
RAM	24GB

Table 5.4: Hardware and runtime configuration for the performance evaluation.

The two modes differ in which metrics they compute. In Quick mode the most expensive metrics are sampled or skipped on large graphs: of the sixty-two cities, the forty-one largest were processed on the approximate path, where exact betweenness centrality is not computed, while the twenty-one smaller cities were computed exactly even in Quick mode. In Full mode the smaller cities still finish quickly, but the very largest networks – those with hundreds of thousands of nodes – take much longer, because exact closeness and betweenness need all-pairs shortest-path work whose cost grows steeply with size. The shortest-path and Brandes computations are parallelised across cores, and analysis runs on a background thread so the interface stays usable while a batch runs.

The intended operating point is that Quick mode keeps comparison across the whole dataset fast, while Full mode is for research runs where the exact figures are worth the wait.

5.7 Use Case Walkthroughs

The three walkthroughs below are *illustrative scenarios* tracing the main workflows; they are not a user study. Each was carried out by the author, with the personas only framing the typical intent behind each path.

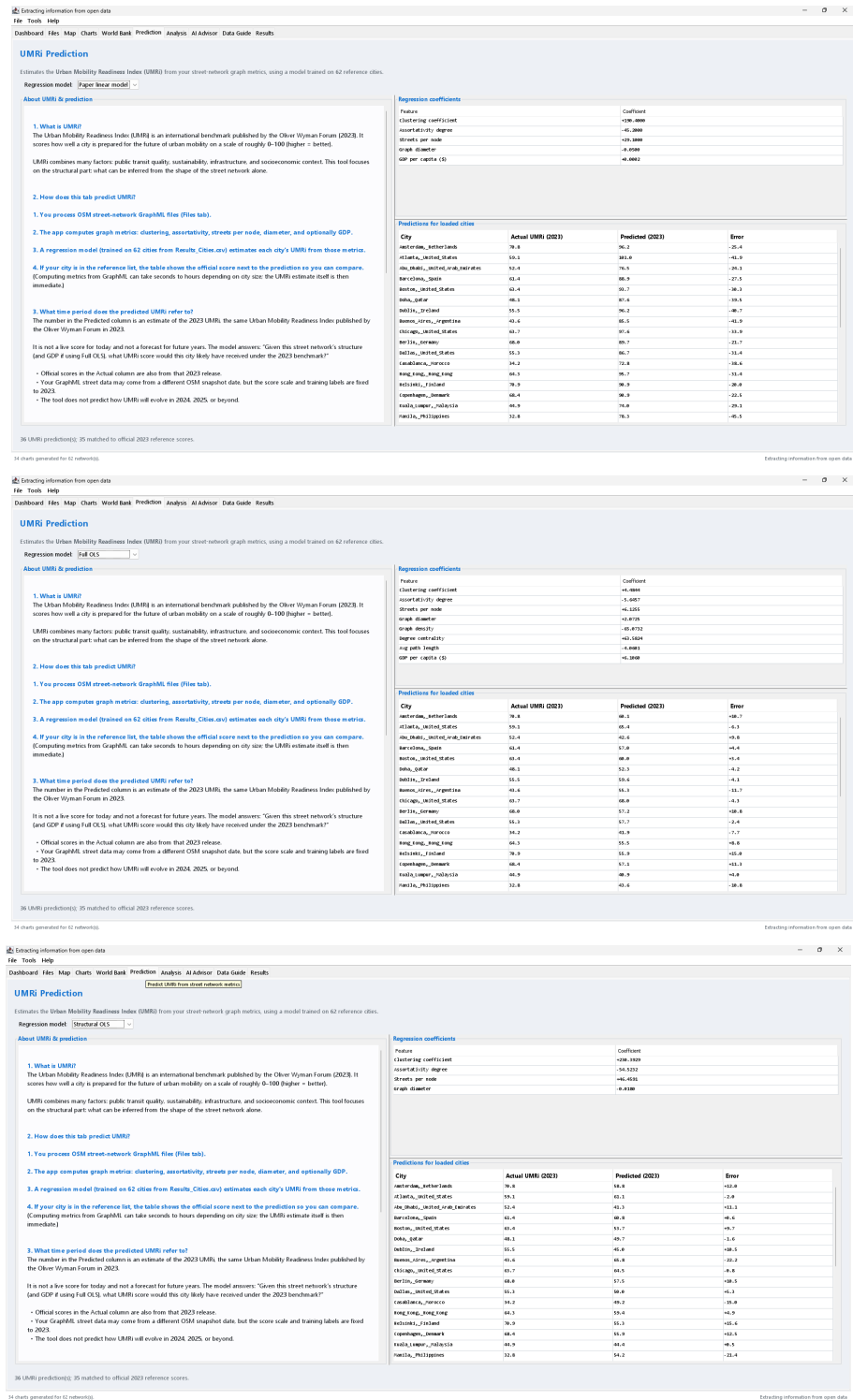


Figure 5.1: The Prediction tab under the three models compared in Table 5.3: structural OLS (top), full OLS (middle), and the fixed-coefficient paper linear model (bottom), each showing the regression coefficients and the per-city actual, predicted, and error columns.

Measurement	Quick mode	Full mode
Mean time per city	~20 s	~5.7 min
Median time per city	~17 s	~88 s
Largest city (Cape Town, ~400k nodes)	~60 s [†]	≥1 h [†]
Smallest city (~3k nodes)	~5 s	~2 s
Total time, full dataset (62 cities)	~21 min	~5.9 h

Table 5.5: Running times over the full dataset in Quick and Full mode. Values marked [†] are measured on the test machine; remaining values are projected from a power-law fit on node count, anchored on the measured cities (Amsterdam, Barcelona, Cape Town). Totals assume sequential execution.

5.7.1 Use Case 1: Researcher – Reproducing the Reference Metrics

Here the application is pointed at the folder of sixty-two GraphML files, all cities selected, Full analysis chosen, and the batch processed. When it finishes, the Results table holds the structural metrics for every city, exported to CSV in one click and compared against the reference dataset to reproduce the published metrics on the matched cities.

5.7.2 Use Case 2: Student – Comparing Familiar Cities

Here a handful of well-known cities are loaded in Quick mode and a side-by-side bar chart of clustering coefficient and streets per node is generated from the Charts tab. The chart makes clear that the more compact, grid-like cities score higher on both, and the Analysis tab’s report explains the link to mobility readiness. The whole thing takes a few minutes and needs no code.

5.7.3 Use Case 3: Analyst – Estimating Readiness for a City

Here a single city’s network is examined, its country’s World Bank indicators (including GDP per capita) are fetched, and the Prediction tab is opened. The application estimates the city’s UMRi, SMi, and PTi from its structural metrics and economic context and reports the model’s cross-validated error, so the analyst can judge how much to trust the estimate.

5.8 Limitations Observed During Testing

A few limitations came up during evaluation. Exact centrality on the largest networks is slow in Full mode, and the Quick-mode approximation, while fast, drops betweenness and clustering on the biggest graphs, leaving those cities with fewer exact metrics for prediction. Matching cities to their reference entries and World Bank country codes also relies on parsing filenames, which can fail for ambiguous names. These are discussed further in Chapter 6.

Chapter 6

Discussion

This chapter interprets the evaluation results of Chapter 5 against the objectives stated in Chapter 1, considers how the application relates to the reference study and to existing tools, reflects on the principal design trade-offs, and examines the limitations observed during testing.

6.1 Achievement of Objectives

The primary objective was to turn the single published analysis of the reference study into a reusable application that loads arbitrary city networks, computes the mobility-relevant metrics, and estimates readiness. The evaluation indicates that this objective has been met. All sixty-two reference cities were processed end to end, and the application’s independently computed metrics agree closely with the published values on the quantities that can be checked exactly – a Pearson correlation of 0.997 on graph diameter and 0.78 on clustering coefficient across the matched cities. This is the key result: it shows that the re-implementation reproduces the reference computation on those quantities rather than approximating it loosely, which is what licenses using the application in place of the original Python analysis.

The prediction objective was also met. The fixed-coefficient model reproduces the reference regression equation, and the run-time OLS models add a cross-validated estimate of out-of-sample accuracy. Reporting the cross-validated R^2 and error alongside each estimate is a deliberate improvement over a bare prediction: it communicates uncertainty, which matters when the output might inform a planning discussion. The socio-economic enrichment objective was met through the World Bank integration, and the accessibility objective through the tabbed Swing interface backed by a scripting-friendly command-line mode.

6.2 Relationship to the Reference Study and Existing Tools

The reference study [SPHA24] established the relationships between street-network structure, economic context, and mobility readiness, but delivered them as analysis rather than as a tool. This thesis contributes the missing artefact. Where the reference work computed metrics once, for one dataset, in notebooks, the application

computes the same metrics on demand for any GraphML network and packages comparison, mapping, enrichment, prediction, and reporting around them. The agreement between the two on shared cities is precisely what allows the application to stand in for the original analysis while extending its reach.

Relative to general-purpose tooling, the application occupies a niche that neither network libraries nor visualisation platforms fill. Libraries such as OSMnx [Boe17] or JGraphT [JGr26] compute metrics but require programming and offer no mobility interpretation, no corpus of ranked cities to compare against, and no readiness estimate. Visualisation platforms can draw charts but understand neither street networks nor mobility indices. By bundling all of the above behind a single interface, the application makes the reference study’s results usable for people who would not otherwise reproduce them.

6.3 Design Trade-offs

6.3.1 Quick versus Full Analysis

The most consequential design decision is the two-mode analysis strategy. Exact centrality requires all-pairs shortest-path computation, whose cost grows steeply with network size; on the largest cities in the dataset this is prohibitively slow for interactive use. Quick mode resolves the tension by sampling source vertices for closeness and path-length statistics and skipping betweenness and, for the very largest graphs, clustering, while Full mode computes everything exactly. The trade-off is explicit and surfaced to the user through an approximation flag on the results, rather than hidden. This keeps the common case – comparing many cities – responsive across the whole dataset, while preserving exact computation for research. The cost is that approximate cities carry fewer exact metrics into the prediction step.

6.3.2 Re-implementation versus Reuse

Rather than calling OSMnx or reusing the reference notebooks, the metrics were re-implemented on top of JGraphT in pure Java. This keeps the application a single self-contained desktop program with no Python dependency, simplifies distribution, and gives full control over the concurrency model. The price is the effort of re-implementation and the need to validate it against the reference – which is exactly why the correctness comparison of Chapter 5 matters: it is the evidence that the independent implementation is trustworthy.

6.3.3 Run-time Regression versus Fixed Coefficients

Offering both a fixed-coefficient model and run-time OLS models is a trade-off between fidelity to the published analysis and methodological honesty. The fixed model reproduces the reference regression exactly but reports an in-sample fit that overstates predictive accuracy; the OLS models are refitted and cross-validated so their reported error reflects genuine out-of-sample performance. The direct comparison of the three methods in Chapter 5 sharpened this trade-off: the refitted models predicted the official scores to within a mean absolute error of roughly seven to nine index points, whereas the fixed-coefficient model, applied to the independently

re-computed metrics, over-predicted every city by around thirty points. The lesson is that published coefficients are inseparable from the precise metric definitions and units on which they were estimated, so a faithful re-implementation must refit rather than transplant them. Providing both models lets the user reproduce the paper’s equation for reference while relying on the cross-validated models for an accurate, scale-correct estimate.

6.3.4 Local Caching and Offline Operation

Caching World Bank responses to disk and supporting an offline flag trades freshness for resilience. Socio-economic indicators change slowly, so serving them from cache is almost always correct and keeps the application usable without network access, at the small cost of needing an occasional cache refresh.

6.4 Limitations and Mitigations

Three limitations stand out. First, exact centrality on the largest networks is slow; the mitigation already in place is Quick mode, and a fuller fix would be to adopt approximate-betweenness algorithms that bound error while remaining fast even on Full runs. Second, the Quick-mode approximation leaves the biggest cities with fewer exact metrics, which slightly weakens their predictions; this could be addressed by computing exact clustering and betweenness for those cities as a background task that upgrades the metrics after the initial fast pass. Third, the matching of cities to reference entries and country codes relies on parsing filenames, which is brittle for non-standard names; a small editable mapping table, or matching on coordinates rather than names, would make this more robust. None of these limitations affects the core result that the application reproduces the reference metrics and predictions on the standard dataset.

6.5 Implications

The larger implications suggest that there is a degree of tooling disparity between mobility research and practice. The reference study demonstrated that compact, connected and well funded street networks are better equipped for sustainable mobility practices; this thesis demonstrates that analytical basis for that conclusion can be encapsulated into an application that may be run by anybody possessing a directory of open street-network files, extended to new cities and utilized to generate not just metrics but an interpreted and contextualized assessment of a city’s mobility readiness.

Chapter 7

Conclusions

7.1 Summary

This thesis presented the design, implementation, and evaluation of a Java desktop application that analyses urban street networks from OpenStreetMap-derived GraphML files, computes the graph-theoretic and street-level metrics used in the urban mobility literature, enriches them with World Bank socio-economic indicators, and estimates a city’s Urban Mobility Readiness Index together with its Sustainable Mobility and Public Transit sub-indices. It takes the study of Sierra-Porta and Herrera-Acevedo [SPHA24] as its specification and turns its metrics, reference cities, and regression findings into a reusable, interactive tool.

The main contributions are:

1. **A self-contained application built from a mobility study.** The reference study’s metric computation and predictive modelling, originally done once in Python, were re-implemented in independent Java as a single desktop application with a Model–View–Controller architecture and a service layer for GraphML loading, structural analysis, prediction, World Bank integration, charting, mapping, reporting, and session persistence, with both a graphical and a command-line interface.
2. **An independently validated re-implementation of the structural metrics.** The metrics – centralities, clustering, diameter, entropy, density, assortativity, reciprocity, diversity, and the street-level OSM statistics – are computed on JGraphT, with the expensive shortest-path and betweenness work parallelised across cores and split into Quick (sampled) and Full (exact) modes. On fifty matched cities, the re-implementation reproduced graph diameter at a Pearson correlation of 0.997 and clustering coefficient at 0.78.
3. **A prediction engine for mobility readiness.** The application estimates UMRi, SMi, and PTi from network structure and economic context using three models: a fixed-coefficient model reproducing the reference regression, and two run-time ordinary least squares models evaluated by leave-one-out cross-validation over the sixty-two cities, with the cross-validated R^2 and error shown alongside every prediction so uncertainty stays visible.
4. **Integration of open socio-economic data.** Country-level indicators – GDP per capita, urban population share, CO₂ emissions per capita, and others

- are fetched and cached from the World Bank Data360 API, giving the economic context that the reference study found to be the strongest single correlate of mobility readiness and feeding the GDP-dependent prediction model.
5. **Accessibility and reproducibility.** The tabbed interface makes the whole workflow – load, measure, compare, map, enrich, predict, report – usable without programming, while the command-line batch mode and automatic session persistence support scriptable, reproducible runs and easy resumption of work.

7.2 Future Work

Several directions remain open. The main performance limit is exact betweenness on the largest networks, so the most useful next step is a sampling-based approximate-betweenness algorithm with bounded error, which would let Full mode scale to the biggest cities. Related to this, cities on the Quick path carry fewer exact metrics; a background task that recomputes exact clustering and betweenness for them after the fast pass would improve their metrics, and their predictions, without making the user wait at load time.

A second group of improvements concerns robustness and data. Replacing filename-based matching with coordinate-based matching, or a small editable mapping table, would make the World Bank enrichment and prediction more reliable for cities with ambiguous names. The reference corpus could also grow: the prediction engine is trained on the sixty-two cities of the 2024 ranking, and adding more ranked cities, or scores from newer editions, would enlarge the training base and let the tool place a wider range of cities in context.

Finally, the modelling could be richer. The current prediction uses linear regression, following the reference study, but the three-method comparison in Chapter 5 showed how sensitive a transplanted fixed-coefficient model is to metric definitions, and how much a refitted model with economic context improves accuracy. Regularised or non-linear models, reported the same cross-validated way, could push out-of-sample accuracy further, especially given the non-linear relationships the reference study observed, while keeping the explanatory layer that makes the tool usable by non-specialists.

These results show that the analysis behind a published urban-mobility study can be re-implemented faithfully and packaged into a practical tool. By reproducing the reference metrics to within a high correlation on the quantities that can be checked exactly, predicting readiness with a clearly reported cross-validated accuracy, and wrapping the workflow in an accessible interface, the application turns a single published analysis into a reusable tool for studying urban street-network structure and a foundation for the extensions above.

Bibliography

- [BDV⁺22] Mohamed Badhrudeen, Sybil Derrible, Trivik Verma, Amirhassan Kermanshah, and Angelo Furno. A geometric classification of world urban road networks. *Urban Science*, 6(1):11, 2022.
- [BKK20] Simon Elias Bibri, John Krogstie, and Mattias Kärrholm. Compact city planning and development: Emerging practices and strategies for achieving the goals of sustainability. *Developments in the Built Environment*, 4:100021, 2020.
- [BLM⁺06] S. Boccaletti, V. Latora, Y. Moreno, M. Chavez, and D.-U. Hwang. Complex networks: Structure and dynamics. *Physics Reports*, 424(4–5):175–308, 2006.
- [Boe17] Geoff Boeing. Osmnx: New methods for acquiring, constructing, analyzing, and visualizing complex street networks. *Computers, Environment and Urban Systems*, 65:126–139, 2017.
- [Bra01] Ulrik Brandes. A faster algorithm for betweenness centrality. *Journal of Mathematical Sociology*, 25(2):163–177, 2001.
- [Fas26] FasterXML. Jackson: high-performance json processor for java. <https://github.com/FasterXML/jackson>, 2026.
- [Fre78] Linton C. Freeman. Centrality in social networks: Conceptual clarification. *Social Networks*, 1(3):215–239, 1978.
- [Goo26] Google. Gemini api documentation. <https://ai.google.dev/>, 2026.
- [JC16] Ian T. Jolliffe and Jorge Cadima. Principal component analysis: a review and recent developments. *Philosophical Transactions of the Royal Society A*, 374(2065):20150202, 2016.
- [JGr26] JGraphT Contributors. Jgrapht – a java library of graph theory data structures and algorithms. <https://jgrapht.org/>, 2026.
- [New02] Mark E. J. Newman. Assortative mixing in networks. *Physical Review Letters*, 89(20):208701, 2002.
- [Ope26] OpenStreetMap contributors. Openstreetmap. <https://www.openstreetmap.org/>, 2026.

- [SPHA24] David Sierra-Porta and Daniel D. Herrera-Acevedo. Network structure and urban mobility sustainability: a topological analysis of cities from the urban mobility readiness index. *Preprint, Universidad Tecnológica de Bolívar*, 2024. <https://data.mendeley.com/datasets/gmyt9wrgst/1>.
- [The24] The Apache Software Foundation. Apache commons math 4.0 snapshot. <https://commons.apache.org/proper/commons-math/>, 2024.
- [WH00] Rüdiger Wirth and Jochen Hipp. CRISP-DM: Towards a standard process model for data mining. In *Proceedings of the 4th International Conference on the Practical Applications of Knowledge Discovery and Data Mining*, 2000.
- [Wor24] World Bank Group. Data360 api and world development indicators. <https://data360.worldbank.org/>, 2024.

Appendix A

Appendix - Generative AI Tools Usage

An overview of how generative AI Tools are used in this thesis is presented in Table A.1.

AI Tool	Use case	Used in	Remarks
Claude (Anthropic)	Generate initial state of the art and bibliography list	Chapter 2	Prompts in section A.1
	Code generation	Chapter 4	Details in section A.1
	Text restructuring and editing	Chapters 1–7	Details in section A.1

Table A.1: Overview of generative AI Usage

A.1 Claude (Anthropic) Details

Use Case 1: Generate Initial State of the Art and Bibliography List

1. **Purpose of use:** To obtain an initial list of candidate academic references and a preliminary outline for the state of the art (Chapter 2), covering complex network analysis, street-network metrics, and urban mobility.
2. **Method:** The following prompts were used to seed reference discovery and the section outline for Chapter 2:
 - *"List foundational and recent papers on complex network analysis and street-network metrics for urban mobility."*
 - *"Summarize standard graph centrality and clustering measures and their relevance to transport networks."*

- *"What are the most cited papers that use OSMnx to extract and analyze street networks? Give author, year, and venue for each."*
 - *"Explain how circuitry, intersection density, and streets-per-node have been used as indicators of urban mobility in the literature."*
 - *"Suggest survey papers that compare graph-based metrics for measuring street network resilience and connectivity."*
 - *"Give me a candidate outline for a state-of-the-art chapter covering complex networks, street-network analysis, and sustainable urban mobility indices."*
3. **How the output was examined/verified:** All suggested references were manually verified for existence and accuracy using Google Scholar and the ACM Digital Library. Fabricated or unverifiable citations were discarded. The content of each retained reference was read and independently summarized; no AI-generated text was used verbatim.
 4. **How the output is implemented/used:** The generated list served as a starting point for literature discovery. Final citations, analysis, and writing in Chapter 2 are the author's own work, informed by reading the actual sources.

Use Case 2: Code Generation

1. **Purpose of use:** To assist with generating boilerplate and implementation code for selected components of the Java desktop application described in Chapter 4 (for example, parallel graph traversal scaffolding and Swing panel layout).
2. **Method:** Prompts describing specific functional requirements were used. Generated code was requested incrementally, one component at a time:
 - *"Write a Java method to compute betweenness centrality on a JGraphT directed graph."*
 - *"Generate a Java Swing panel that displays a sortable metrics table."*
 - *"Help me write a streaming GraphML loader in Java that parses node and edge attributes without loading the whole file into memory."*
 - *"Write a JUnit test for a class that parses city location strings into country codes, covering edge cases like missing or malformed input."*
 - *"Refactor this Swing panel so the chart rendering runs off the Event Dispatch Thread and updates the UI safely."*
 - *"Generate a service class that exports a metrics table to CSV and XLSX, given a list of NetworkMetrics objects."*
 - *"Suggest a clean way to structure a spatial index for street segments so nearest-segment lookups on the map are fast."*
3. **How the output was examined/verified:** All generated code was reviewed line by line, tested against sample datasets, and integrated manually into the application's architecture. Code that did not conform to the project's design patterns or produced incorrect results was rewritten or discarded.

4. **How the output is implemented/used:** Accepted code segments were adapted and integrated into the application codebase. All code present in the thesis listing has been reviewed, understood, and is fully owned by the author. AI-generated code was never used without comprehension or testing.

Use Case 3: Text Restructuring and Editing

1. **Purpose of use:** To improve the clarity, flow, and structure of text already written by the author across the thesis (Chapters 1–7), including reorganizing paragraphs, tightening wording, and improving transitions, without introducing new technical claims.
2. **Method:** Author-written drafts were provided to the tool together with restructuring instructions. The following prompts were used:
 - *"Restructure this paragraph so the main result comes first, then the explanation. Keep my wording and meaning, only reorder and tighten."*
 - *"This introduction feels repetitive. Suggest a clearer structure for these paragraphs without adding any new content."*
 - *"Improve the transitions between these three subsections so they read as a continuous argument."*
 - *"Rewrite this passage to be more concise and academic in tone, but do not change the technical claims or add citations."*
 - *"Check whether the order of these methodology steps is logical and suggest a reordering if it is not."*
 - *"Point out where this discussion section is unclear or where sentences are too long, and propose shorter versions."*
3. **How the output was examined/verified:** Every suggested revision was reviewed by the author and accepted, modified, or rejected on a case-by-case basis. The tool was not allowed to introduce new technical statements, results, or references; suggestions that altered meaning were discarded. The final wording was confirmed against the underlying data, code, and sources.
4. **How the output is implemented/used:** Only structural and stylistic suggestions that preserved the author's intended meaning were incorporated. The technical content, arguments, and conclusions remain the author's own work.